Missing Data and Complete Case Analysis

Mechanisms, the five-percent rule and dropping rows

Contents

Missing Data and Complete Case Analysis 2
35.1 What counts as missing 2
35.2 The three mechanisms 2
35.3 Complete-case analysis, defined 4
35.4 The lecture's HR-analytics audit 4
35.5 Three checks before you trust a deletion 5
35.6 The honest trade-off 6
35.7 When CCA is the right choice 6
35.8 Recap, practice and sources 6

CHAPTER 35

Missing Data and Complete Case Analysis

Mechanisms, the five-percent rule, and when dropping rows is honest

> **WHAT THIS CHAPTER BUILDS**
>
> Before any value is filled in, two questions decide everything: why is the value missing, and how much of the data would disappear if it were dropped. This chapter names the three mechanisms, derives what complete-case analysis is allowed to assume, and applies the lecture's HR-analytics audit line by line.

35.1 What counts as missing

A missing value is the absence of an observation where an observation was expected. In pandas that is usually `NaN` for numeric columns, `NaT` for datetime columns, and `None` or `pd.NA` for object columns. `DataFrame.isnull()` detects all of them.

```
df.isnull().sum()            # count per column
df.isnull().mean()           # fraction per column
df.isnull().mean().sort_values(ascending=False)
```

The fraction matters more than the count. A count of 500 is catastrophic in 600 rows and irrelevant in 1,000,000. Start every missing-data audit with the sorted fraction, and read it alongside the column's role: the target, the identifier, and a rarely used attribute deserve different treatment than a central predictor.

Real data also hides missingness behind sentinel values. A `0` in an income column, `-1` in an age column, `"Unknown"` in a city column, or `999` in a phone column are all "missing" in the sense that the number or word is a placeholder rather than an observation. Detect them before any imputer runs:

```
sentinels = {"income": [0], "age": [-1], "city": ["Unknown"]}
for col, vals in sentinels.items():
    df[col] = df[col].replace(vals, np.nan)
```

If the sentinel and the real value are indistinguishable, the feature is corrupted from the model's point of view. Converting sentinels is a data-cleaning step that belongs before train/test splitting, because it is deterministic and stateless.

> **Missingness is information about collection**
>
> A blank is rarely a fact about the person or object. It is a fact about how the row was recorded: a skipped form field, a sensor that failed, a value withheld on purpose. That is why the **reason** for the gap, not the gap itself, determines the repair.

35.2 The three mechanisms

The lecture states the taxonomy plainly: there are three categories of missing data.

Missing completely at random (MCAR)

The probability that a value is missing does not depend on any observed or unobserved data. The data simply was not collected for some rows: a survey page failed to load, a batch of samples was lost. If you could see the missing values, knowing them would not change the probability that they are missing.

Missing at random (MAR)

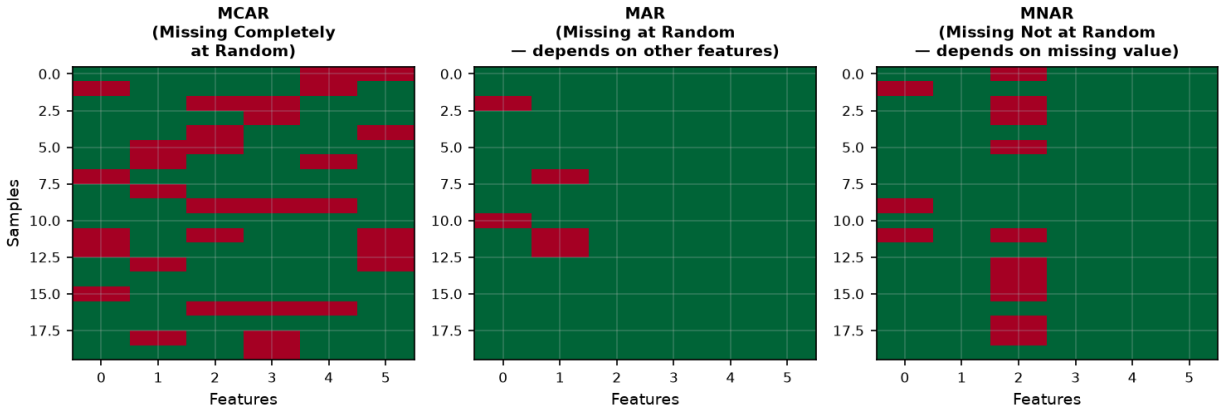
The probability that a value is missing depends on other columns that **are** observed. Someone skips the weight field because they are self-conscious about it, or skips the age field because they do not want to disclose it; the choice is visible through other recorded attributes. Formally, the missingness of X is independent of X itself conditional on the observed data: $P(X \text{ missing} \mid X, Z) = P(X \text{ missing} \mid Z)$, where Z is the observed part. The missing value can therefore be predicted from the other columns.

Missing not at random (MNAR)

The probability that a value is missing depends on the value itself, or the value was removed deliberately with no usable relationship to the other columns. High earners omitting income, or a censoring rule that hides exactly the extreme cases, belongs here. The lecture's phrasing: the data was removed intentionally, and there is not much relationship between the other columns and the missing one, so you cannot predict it.

These definitions are assumptions about a data-generating process, not properties you can prove from the table. You argue for them from how the data was collected. A Little's MCAR test or a model that predicts missingness from the observed columns can provide evidence, but no test can certify MAR versus MNAR, because MNAR is precisely the case where the relevant variable is unobserved.

Mechanism	Missingness depends on	What it licenses
MCAR	Nothing observed or unobserved	Complete-case analysis; simple mean/median
MAR	Observed columns only	Imputation from observed columns: regression, MICE
MNAR	The missing value itself	No standard method is valid without extra modelling



Three missingness patterns. MCAR scatters blanks without structure; MAR ties blanks to an observed column; MNAR ties them to the hidden value itself.

The mechanisms also explain why the imputation chapters work at all. Mean imputation asks only that the gaps be ignorable; MICE in Chapter 40 explicitly assumes MAR, because it uses the other columns to predict the missing one. If the data is MNAR, no amount of machinery in the following chapters repairs the bias; the missingness itself must be modelled.

35.3 Complete-case analysis, defined

Complete-case analysis (CCA) is the simplest repair: delete every row that has a missing value in any of the columns you intend to use. It is also called listwise deletion.

$$x_i \text{ is kept iff } x_{ij} \text{ is observed for every used column } j$$

In pandas:

```
used = ["city_development_index", "enrolled_university",
        "education_level", "experience", "training_hours"]

df_cca = df.dropna(subset=used)
print(len(df_cca) / len(df))     # completeness ratio
```

`dropna` has four controls: `subset`, the columns to consider; `how="any"` (default) or `how="all"`, whether one gap or all gaps kills the row; `thresh`, the minimum number of non-null values to keep a row; and `axis`, rows or columns. CCA with `subset` is the ordinary case.

The completeness ratio is the single most important number. It tells you the price of the deletion before you run anything.

35.4 The lecture's HR-analytics audit

The lecture runs the audit on a real HR-analytics dataset of about 19,000 job applicants, one row per applicant, with columns for enrollee ID, city, city development index, gender, relevant experience, enrolled university, education level, major discipline, experience, company size, company type, training hours, and the hiring target.

First the missing rates are printed. The picture the lecture draws:

- ID-like columns, city, relevant experience and the target have no missing values.
- `gender` has a large gap: running CCA on gender alone would drop roughly 23% of the rows.
- `company_size` is missing for about 30.8% of rows and `company_type` for about 32.2%.
- `city_development_index`, `enrolled_university` and `education_level` are each around 2%.

- `experience` is around 0.3%, `training_hours` around 4%.

The decision follows from those numbers. CCA on the three heavy columns would throw away a third of the dataset, which the lecture treats as unacceptable. The five columns at or below the five-percent threshold become the CCA candidates: `city_development_index`, `enrolled_university`, `education_level`, `experience`, `training_hours`.

```
missing = df.isnull().mean()
low_missing = missing[missing <= 0.05].index.tolist()
# remove identifier/target columns from the list if they appear

df_cca = df.dropna(subset=low_missing)
print(len(df_cca), len(df), len(df_cca) / len(df))
```

On the lecture's run, the frame printed before deletion was 19,020 rows and the frame after dropping incomplete rows among the selected columns was around 17,900; the ratio printed by `len(df_cca) / len(df)` is therefore close to 0.94. Only a single-digit percentage of rows disappeared, which is the only situation in which CCA is worth considering.

> **The five-percent rule is a heuristic, not a theorem**
>
> There is no law that says 5% is safe and 5.1% is not. The lecture is explicit about this: no rule exists, but people generally work with five percent. The reasoning behind it is that a small deletion is unlikely to move a distribution, while a large one usually is. What you must never do is apply CCA without checking what disappeared.

35.5 Three checks before you trust a deletion

1. Numeric distributions. Plot a histogram or density curve for each numeric column before and after the deletion. Overlapping curves are a green flag: the rows that survived look like the rows that were there.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
df["training_hours"].plot(kind="kde", ax=axes[0], title="before")
df_cca["training_hours"].plot(kind="kde", ax=axes[1], title="after")
```

The lecture does exactly this for `training_hours`, `city_development_index` and `experience`, overlaying the before and after curves in two colours. The curves overlap, small local differences aside, which supports the claim that the missing rows were MCAR with respect to those columns.

2. Categorical proportions. For each categorical column, compare `value_counts(normalize=True)` before and after. The lecture's example is `enrolled_university`, with three levels: no enrollment, full-time course, part-time course. The proportions before the deletion were around 0.72, 0.194 and 0.06; after the deletion they moved only in the third decimal place. That stability is the evidence.

The lecture sets a concrete alert level: a shift of a few points in the third decimal is noise, while a jump such as 0.19 or 0.25 in a proportion is a red flag that the deleted rows were systematically different. The same check on `education_level` (graduate, master, high school, PhD, primary school) showed each level holding roughly its share after deletion.

3. The target. Compare the target's distribution or class balance before and after. If the positive class drops from 35% to 22%, the deletion has silently changed the problem you are solving.

35.6 The honest trade-off

The lecture's advantages of CCA are two, and both are real:

- **Simplicity.** Call `dropna` and the job is done. No data manipulation, no fitted parameters, no new columns.
- **Distribution preservation among the kept rows.** If the missingness is MCAR, deleting rows is equivalent to deleting a random subsample, and a random subsample preserves the distribution.

The disadvantages are the reasons CCA is rarely the final answer:

- **You may lose a large fraction of the original data.** The lecture's extreme example: a column that is 98% missing. Deleting the rows leaves almost nothing; the sensible move there is to delete the **column**.
- **Bias when the data is not MCAR.** If missingness correlates with the value, the surviving rows are a selected sample, and every statistic computed from them is shifted.
- **The production problem, which the lecture calls the most important point.** Your model was trained only on complete rows. On a live server, incoming rows arrive with gaps, and the model has never learned how to handle them. It has no mechanism for a missing input, so the service either fails or silently drops customers. This is why most practitioners impute rather than delete.

> **The deployment argument is the decisive one**
>
> CCA makes a promise the real world will not keep: that every future row will be complete. Imputation is usually preferred not because it is more accurate on the training set, but because it gives the deployed model a defined behaviour when a value is missing.

35.7 When CCA is the right choice

Two conditions, both required:

1. The data is missing completely at random, argued from the collection process, not guessed from the table.
2. The missingness in every used column is small enough that the deletion does not move the distributions; the convention is roughly 5% per column.

If a column is nearly all missing, drop the column instead of the rows. If the mechanism is MAR or MNAR, impute instead of deleting. And whatever you choose, the choice is a modelling decision: record the completeness ratio, the distributions you compared, and the columns you excluded, so the next reader knows what the dataset looked like before the rows vanished.

35.8 Recap, practice and sources

The chain is: audit the sorted missing fractions; convert sentinel values to real missing values; argue for MCAR, MAR or MNAR from the collection process; delete rows only under MCAR and only at small rates; compare numeric distributions, categorical proportions and the target before and after; remember that a trained-on-complete model cannot serve incomplete rows.

Practice

1. Print `df.isnull().mean().sort_values(ascending=False)` on any dataset. Which columns would fail the 5% test?
2. Run CCA on the three heaviest columns of the HR dataset and compute `len(df_cca) / len(df)`. Now run it on the five lightest. Compare.

3. Compare `enrolled_university` proportions before and after CCA. Are the shifts in the third decimal?
4. Construct an MCAR example by deleting 5% of values at random, and an MNAR example by deleting the largest 5%. Run the same diagnostics on both. Which one is detectable?
5. Explain in one paragraph why CCA creates a production problem that no amount of training accuracy reveals.

Sources and scope

Video 35, “Missing Data / Complete Case Analysis”, CampusX 100 Days of Machine Learning. The lecture covers the MCAR/MAR/MNAR taxonomy, the five-percent convention, the advantages and disadvantages of CCA, the production objection, and a full HR-analytics demonstration with distribution and proportion checks. The conditional-independence notation and the note on Little’s test are supplementary formalisations of the lecture’s stated assumptions.